Ders 26 Genişletilmiş Notu: Karşılaştırmasız Sıralama: Counting Sort, Radix Sort ve Sıralama Algoritmalarının Karşılaştırılması

Ders 26 Genişletilmiş Notu: Karşılaştırmasız Sıralama: Counting Sort, Radix Sort ve Sıralama Algoritmalarının Karşılaştırılması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Karşılaştırma Yapmadan Sıralamak
- 2. Counting Sort (Sayarak Sıralama)
- 3. Comparison Counting Sort (Karşılaştırmalı Sayma)
- 4. Counting Sort Nerede İşe Yarar?
- 5. Distribution Counting Sort (Dağılım Sayımı)
- 6. Radix Sort: Delikli Kartlardan Gelen Bir Algoritma
- 7. LSD Radix Sort
- 8. MSD Radix Sort
- 9. Sıralamanın Tanımı ve Önemi
- 10. Sıralama Algoritmalarını Karşılaştırma Ölçütleri
- 11. Tasarım Yaklaşımlarına Göre Sıralama Algoritmaları
- 12. Karşılaştırma Tablosu ve Algoritma Seçimi
- Örnek Sorular

1. Karşılaştırma Yapmadan Sıralamak

Bu ders, Veri Yapıları ve Algoritmalar dersinin 14. haftası ve dönemin son teorik dersiydi. Konu yine sıralamaydı, ama hoca sıralamaya farklı bir açıdan bakmak istedi. İki soruyu ele aldı: Farklı mantıklarla sıralama yapılabilir mi? Sıralama algoritmaları nasıl karşılaştırılır ve hangi durumda hangisini kullanacağımıza nasıl karar veririz?

Problem şuydu: Bir diziyi, örneğin tam sayılardan oluşan bir diziyi, hiçbir karşılaştırma yapmadan sıralamak istiyoruz. Yani “bu eleman şundan büyük mü, küçük mü?” diye soran bir `if` kullanmayacağız. Hoca bunu şöyle bir senaryoyla anlattı: Öyle bir mimaride çalışıyoruz ki karşılaştırma yapmak çok maliyetli. Bu durumda karşılaştırmasız sıralama yapabilir miyiz?

Hocaya göre bunun bir çözümü var. Bu çözüm kaynaklarda saymayla sıralama, doğrusal sıralama (linear sort) ya da sayarak doğrusal sıralama gibi farklı adlarla geçer. Derste kullanılan adlar counting sort ya da sorting by counting oldu.

2. Counting Sort (Sayarak Sıralama)

2.1 Algoritma

Hoca algoritmayı slayttaki sözde kod (pseudocode) üzerinden anlattı. Uzaktan derste akış diyagramı çizmek yerine sözde kodla anlatmanın daha rahat olduğunu söyledi. Sözde kodda

toplam dört `for` döngüsü vardır ve bunların hiçbirinin içinde elemanlar arasında karşılaştırma yapılmaz. Counting sort, elemanların anahtar değerlerini yalnızca sayarak sıralar.

Elimizde değerleri 0 ile k arasında olan n elemanlı bir A dizisi olsun. Algoritmanın adımları şunlardır:

- **Sıfırlama:** Her değerden kaç tane olduğunu saymak için bir sayaç dizisi (count) C oluşturulur ve hepsi sıfırlanır. Bu döngü $O(k)$'dır.
- **Sayma:** Dizinin her $A[j]$ elemanı için $C[A[j]]$ bir artırılır. Bu döngü $O(n)$'dir. Hoca, n boyutlu döngülerde j, k boyutlu döngülerde i yineleyicisini (iterator) kullandığını özellikle belirtti.
- **Birikimli toplam:** C dizisinin her gözü bir öncekiyle toplanır. Böylece her göz, o değerin sıralı dizide nerede biteceğini gösterir. Bu döngü $O(k)$'dır.
- **Yerleştirme:** Girdi dizisinde sondan başa ($n-1$'den 0'a) gidilir. Her $A[j]$ için $C[A[j]]$ bir azaltılır ve $A[j]$, çıktı dizisi B'nin bu indisine yazılır. Bu döngü $O(n)$'dir.

```
COUNTING-SORT(A, B, k)
  C[0..k] = 0                        // O(k)
  her j = 0..n-1 için C[A[j]]++      // O(n): sayma
  her i = 1..k için C[i] += C[i-1]   // O(k): birikimli toplam
  her j = n-1..0 için                // O(n): sondan başa yerleştirme
    C[A[j]]--
    B[C[A[j]]] = A[j]
```

Hoca birikimli toplamı şöyle okumayı önerdi: "Benden küçük kaç tane değer var?" ya da "Ben kaç taneden daha büyüğüm?" Örneğin bir elemandan küçük 10 tane eleman varsa, bu eleman sıralı dizide 11. gözde (0'dan sayınca indis 10) durmalıdır. Algoritmanın yaptığı tam olarak budur.

Aşağıdaki C kodu, slayttaki sözde kodun karşılığıdır. Derste C kodu yazılmadı; kod `gcc -Wall` ile derlenip dersteki örnekle denendi.

```c
#include <stdlib.h>

/* A[0..n-1] değerleri 0..k aralığında, sonuç B'ye yazılır */
void counting_sort(const int A[], int B[], int n, int k)
{
    int *C = calloc(k + 1, sizeof(int));     /* C[0..k] = 0 */
    for (int j = 0; j < n; j++)              /* sayma */
        C[A[j]]++;
    for (int i = 1; i <= k; i++)             /* birikimli toplam */
        C[i] += C[i - 1];
    for (int j = n - 1; j >= 0; j--) {       /* sondan başa yerleştirme */
        C[A[j]]--;
        B[C[A[j]]] = A[j];
    }
    free(C);
}
```

2.2 Örnek: 3 5 2 2 3 1 4

Hoca tahtaya 3 5 2 2 3 1 4 dizisini yazdı ve değerler için 1'den 5'e kadar gözleri olan bir sayaç dizisi çizdi.

Sayma. Dizi baştan sona okunur: 3. gözü bir artır, 5. gözü bir artır, 2. gözü bir artır, 2. gözü bir daha artır, 3. gözü bir daha artır, 1. gözü bir artır, 4. gözü bir artır. Sonuçta:

Değer	1	2	3	4	5
Sayı	1	2	2	1	1

Birikimli toplam. Her göz bir öncekiyle toplanır: $1, 1+2=3, 3+2=5, 5+1=6, 6+1=7$.

Değer	1	2	3	4	5
Birikimli C	1	3	5	6	7

Bu dizi, her değerin sıralı dizide nerede bittiğini gösterir. 1'ler ilk sırada, 2'ler 3'e kadar, 3'ler 5'e kadar, 4 altıncı, 5 yedinci sırada biter.

Yerleştirme. Girdide sondan başa gidilir. Hoca ilk adımı gösterdi: Son eleman 4'tür. 4'ün gözünde 6 vardır. Bir azaltılır, 5 olur ve 4 değeri çıktının 5 numaralı indisine yazılır. Diğer adımlar aynı şekilde ilerler (ara adımlar eklendi):

İşlenen $A[j]$	$C[A[j]]$ (önce)	Yazılan indis	Çıktı B
$A[6] = 4$	6	5	_ _ _ _ _ 4 _
$A[5] = 1$	1	0	1 _ _ _ _ 4 _
$A[4] = 3$	5	4	1 _ _ _ 3 4 _
$A[3] = 2$	3	2	1 _ 2 _ 3 4 _
$A[2] = 2$	2	1	1 2 2 _ 3 4 _
$A[1] = 5$	7	6	1 2 2 _ 3 4 5
$A[0] = 3$	4	3	1 2 2 3 3 4 5

Sonuç `1 2 2 3 3 4 5` olur. Hoca bunu indislerle özetledi: Bir tane 1 sıfırıncı indiste, iki tane 2 birinci ve ikinci indislerde, iki tane 3 üçüncü ve dördüncü indislerde, 4 beşinci indiste, 5 altıncı indiste durur. Her değer, kaçıncı indisten başlayıp kaç yer kaplayacağını birikimli dizi sayesinde bilir.

Sondan başa gitmenin önemi, eşit elemanların girdideki sırasının korunmasıdır. Tablodaki iki 3'e bakın: $A[4]$ indis 4'e, $A[0]$ indis 3'e yazıldı. Yani girdide önde olan 3, çıktıda da önde kaldı. Hoca dersin ilerleyen kısmında bu özelliğe (kararlılık) tekrar döndü.

2.3 Karmaşıklık ve Bedeli

Dört döngünün toplamı $k + n + k + n$ olur. Hoca bunu $O(2n + k)$ diye yazdı, sabitleri atınca $O(n + k)$ dedi. Yani karşılaştırmasız sıralama doğrusal (linear) zamanda çalışır.

> **Ders sonrası not:** Dört döngünün toplamı $2n + 2k$'dır, $2n + k$ değil. Sonuç yine $O(n + k)$'dır.

"Madem doğrusal, her şeyi bununla sıralayalım, neden $n \log n$ algoritmalarla uğraşalım?" diye düşünülebilir. Hoca bunun bedelini açıkladı: çok fazla yer kaybı vardır.

- k boyutlu bir sayaç dizisi gerekir.
- Dizi yerinde (in-place) sıralanamaz, girdi dizisinin üzerine yazılamaz. Çıktı başka bir yerde oluşturulur. Yani girdiyle aynı boyutta bir çıktı alanı daha gerekir.

Kazanç ise hiç `if` kullanmamaktır. Karşılaştırmanın çok maliyetli olduğu bir mimaride, yalnızca dönerek ve sayarak hangi elemandan kaç tane olduğu ve her elemanın hangi indisten başlayıp kaç yer kaplayacağı bulunur.

> **Ders sonrası not:** Hoca k'yı "tekil (unique) eleman sayısı" diye tanımladı. Sayaç dizisinin boyu aslında değer aralığının boyudur ($u - l + 1$). Örneğin dizide yalnızca 1 ve 1000000 değerleri varsa farklı değer sayısı 2'dir, ama 1000000 boyutlu bir sayaç dizisi gerekir.

2.4 Soru: Döngünün Kendisi Bir Karşılaştırma Değil mi?

Soru: `for` döngüsünün kendisi bir kontrol içermiyor mu? Sonuçta döngü de bir `if` ifadesi barındırıyor.

Cevap: Hoca önce sesin kesik geldiğini söyledi ve sorunun yazılmasını istedi. Sonra sözde kodu tekrar gezdi: Birinci döngüde atama, ikincide sayma, üçüncüde birikimli toplam, dördüncüde çıktıya yazma vardır, hiçbirinde `if` yoktur. Döngünün bitiş koşulunun kontrol edildiğini

kabul etti, ama bunu döngünün bir parçası saydıklarını, ayrıca bir `if` olarak değerlendirmediklerini söyledi. Adım sayısı baştan belli olduğu için bir mimari `for` döngüsünü farklı biçimde de kurgulayabilir.

> **Ek bilgi (derste anlatılmadı):** Karşılaştırmalı sıralama algoritmalarının (merge sort, heap sort, quicksort vb.) en kötü durumda $\Omega(n \log n)$ karşılaştırma yapması gerektiği kanıtlanmıştır. Bu alt sınır yalnızca elemanların anahtarlarını birbiriyle karşılaştırarak sıralayan algoritmalar için geçerlidir. Counting sort ve radix sort anahtarları birbiriyle karşılaştırmaz, değerleri indis olarak kullanır. Bu yüzden bu alt sınıra bağlı değildir ve $O(n+k)$ gibi doğrusal sürelere inebilir. Döngü sayacının kontrolü bu sayıma girmez.

3. Comparison Counting Sort (Karşılaştırmalı Sayma)

3.1 Fikir

Hoca daha basit bir alternatif olarak karşılaştırmalı sayma sıralamasını (comparison counting sort) verdi ve "Ne oldu? `if` geri geldi." dedi. Bu algoritmada her eleman için şu sayılır: Dizinin geri kalanında ben kaç elemandan büyüğüm, kaç eleman benden büyük? İki eleman tek bir karşılaştırmayla karşılaştırılır ve sonuca göre ikisinden birinin sayacı artırılır.

- $A[i] < A[j]$ ise büyük olan $A[j]$'nin sayacı artar.
- Değilse $A[i]$'nin sayacı artar.

Dış döngü 0'dan $n-2$'ye, iç döngü $i+1$'den $n-1$'e gider. Yani her eleman yalnızca kendinden sonrakilerle karşılaştırılır; hoca buna üçgen tarama dedi. Sonunda her elemanın sayacı, o elemandan küçük eleman sayısını, yani çıktıdaki indisini verir. Çıktı dizisi S'de $S[\text{Count}[i]] = A[i]$ yapılır.

```
COMPARISON-COUNTING-SORT(A)
  Count[0..n-1] = 0
  her i = 0..n-2 için
    her j = i+1..n-1 için
      eğer A[i] < A[j] ise Count[j]++
      değilse Count[i]++
  her i = 0..n-1 için S[Count[i]] = A[i]
```

C karşılığı (derste C kodu yazılmadı; `gcc -Wall` ile denendi):

```c
#include <stdlib.h>

void comparison_counting_sort(const int A[], int S[], int n)
{
    int *Count = calloc(n, sizeof(int));
    for (int i = 0; i <= n - 2; i++)
        for (int j = i + 1; j <= n - 1; j++) {
            if (A[i] < A[j])
                Count[j]++;
            else
                Count[i]++;
        }
    for (int i = 0; i < n; i++)
        S[Count[i]] = A[i];
    free(Count);
}
```

3.2 Örnek: `62 31 84 96 19 47`

Başlangıçta bütün sayaçlar sıfırdır.

$i = 0$ **(62).** 62, kendinden sonraki her elemanla karşılaştırılır:

- 62 ile 31: 62 büyük, 62'nin sayacı 1 olur.
- 62 ile 84: 84 büyük, 84'ün sayacı 1 olur.

- 62 ile 96: 96 büyük, 96'nın sayacı 1 olur.
- 62 ile 19: 62 büyük, 62'nin sayacı 2 olur.
- 62 ile 47: 62 büyük, 62'nin sayacı 3 olur.

Count dizisi `3 0 1 1 0 0` olur. Hocanın yorumu: 62 kendinden sonraki elemanların 3 tanesinden büyüktür; 84 ve 96 ise şimdiye kadar incelenen 62'den büyüktür.

$i = 1$ **(31).** 31 ile 84: 84'ün sayacı 2 olur. 31 ile 96: 96'nın sayacı 2 olur. 31 ile 19: 31'in sayacı 1 olur. 31 ile 47: 47'nin sayacı 1 olur. Count dizisi `3 1 2 2 0 1` olur.

$i = 2$ **(84).** 84 ile 96: 96'nın sayacı 3 olur. 84 ile 19: 84'ün sayacı 3 olur. 84 ile 47: 84'ün sayacı 4 olur. Count dizisi `3 1 4 3 0 1` olur.

$i = 3$ **(96).** 96 hem 19'dan hem 47'den büyüktür. Sayacı önce 4, sonra 5 olur. Count dizisi `3 1 4 5 0 1` olur.

$i = 4$ **(19).** 19 ile 47: 47'nin sayacı 2 olur. 19'un sayacı 0 kalır. Count dizisi `3 1 4 5 0 2` olur.

Adım	Count dizisi
Başlangıç	0 0 0 0 0 0
$i = 0$ (62)	3 0 1 1 0 0
$i = 1$ (31)	3 1 2 2 0 1
$i = 2$ (84)	3 1 4 3 0 1
$i = 3$ (96)	3 1 4 5 0 1
$i = 4$ (19)	3 1 4 5 0 2

Son Count dizisi `3 1 4 5 0 2`, her girdinin çıktıdaki indisini gösterir. Hoca farkı şöyle vurguladı: İlk counting sort'ta "ben kaçıncı sıradayım ve benden kaç tane var" diye artımlı bir dağılım kurulmuştu. Karşılaştırmalı sayımda ise her girdinin çıktıdaki indisi doğrudan bulunur.

Yerleştirme: 62 → 3. göz, 31 → 1. göz, 84 → 4. göz, 96 → 5. göz, 19 → 0. göz, 47 → 2. göz. Sonuç `19 31 47 62 84 96` olur.

3.3 Karmaşıklık

İç içe iki döngü nedeniyle karşılaştırma sayısı yaklaşık $n^2/2$'dir. Tam değeri $n(n-1)/2$'dir; bu örnekte $6 \cdot 5/2 = 15$ karşılaştırma yapıldı (ara adım eklendi). Karşılaştırma mantığına dönünce karmaşıklık yine $O(n^2)$ mertebesine çıkar. Ama algoritmanın yaptığı iş öğreticidir: Her eleman için "çıktıda nerede durmalıyım, gerçekte kaçıncı değerim?" sorusunun cevabı bir dizide tutulur.

4. Counting Sort Nerede İşe Yarar?

Hocaya göre counting sort, yapılabilecek en az sayıda indis değişikliğiyle (son döngüdeki yazmalarla) her elemanı sıralı dizideki doğru konumuna koyar. Sıralanacak değerler küçük bir dağılım kümesine sahipse sayaç dizisi küçük olur ve algoritma çok verimli çalışır.

- Çok tekrarlı değerler varsa counting sort çok avantajlıdır.
- `if` kullanmaktan kaçınmak gerekiyorsa counting sort kullanılır.

Hocanın örneği ilkokul öğrencileriydi. Birinci sınıftan dördüncü sınıfa kadar öğrencilerin yaşları 5-6 ile 10-11 arasında değişir. Ama her yaştan çok sayıda öğrenci vardır: 5 yaşında belki 2 milyon, 6 yaşında 3 milyon, 7 yaşında 2,5 milyon öğrenci. Farklı değerler (5, 6, 7, 8, ...) çok azdır, ama her biri çok fazla tekrar eder, yani frekansları yüksektir. Böyle bir veriyi yaşa göre sıralamak için counting sort çok doğru ve iyi bir çözümdür.

5. Distribution Counting Sort (Dağılım Sayımı)

5.1 Fikir

Hoca ilk counting sort'u biraz değiştirerek dağılım sayımı (distribution counting sort) adlı yönteme geçti. Her elemanın tekrar sayısı, yani frekansı bir F dizisinde tutulur. Bu, az önceki

count dizisinin aynısıdır.

Çok tekrar eden değerler sıralandığında yerleri bellidir. Değerler l ile u arasındaysa:

- En küçük değer l'ye eşit elemanlar (örneğin 5 yaşındakiler) 0 ile $F[0] - 1$ arasındaki konumlara yerleşir.
- $l + 1$'e eşit olanlar (6 yaşındakiler) $F[0]$ ile $F[0] + F[1] - 1$ arasına yerleşir.
- Bu böyle devam eder.

Bu, ilk counting sort'taki birikimli toplamın ta kendisidir. İstatistikte birikmiş frekans toplamlarına dağılım (distribution) denir; algoritmanın adı buradan gelir.

5.2 Algoritma

Girdi, $l \leq u$ olmak üzere $[l, u]$ aralığında n tane tam sayıdır. l ve u alt (lower) ve üst (upper) sınırlardır; öğrenci örneğinde 5 ve 10 yaş gibi. Amaç bunları artan sırayla sıralamaktır. Hoca bu sefer sayaç dizisine "distribution"ın baş harfiyle D dedi.

- D sıfırlanır.
- Her $A[i]$ için $D[A[i] - l]$ artırılır. l'yi çıkarmak bir indis kaydırmasıdır; böylece l değeri 0. göze düşer.
- D birikimli hale getirilir.
- Sondan başa gidilir. Her $A[i]$ için $j = A[i] - l$ bulunur, $A[i]$ çıktının $D[j] - 1$ indisine yazılır ve $D[j]$ bir azaltılır.

```
DISTRIBUTION-COUNTING-SORT(A, l, u)
  D[0..u-l] = 0
  her i = 0..n-1 için D[A[i] - l]++
  her j = 1..u-l için D[j] += D[j-1]
  her i = n-1..0 için
    j = A[i] - l
    S[D[j] - 1] = A[i]
    D[j]--
```

Hoca bunun dersin başındaki counting sort'un aynısı olduğunu söyledi. Karmaşıklık yine $O(n+k)$'dır. Buradaki k'ya aralık (range) anlamında r de denebilir: l ile u arasındaki değer sayısı. Ek alan olarak bir sayaç dizisi gerekir; farklı sembol sayısı az olduğu için bu önemli değildir. Ama çıktı yine ayrı bir yerde oluşturulur ve ekstra alan ihtiyacı büyüktür.

C karşılığı (derste C kodu yazılmadı; `gcc -Wall` ile denendi):

```c
#include <stdlib.h>

/* l <= A[i] <= u */
void distribution_counting_sort(const int A[], int S[], int n, int l, int u)
{
    int *D = calloc(u - l + 1, sizeof(int));
    for (int i = 0; i < n; i++)
        D[A[i] - l]++;
    for (int j = 1; j <= u - l; j++)
        D[j] += D[j - 1];
    for (int i = n - 1; i >= 0; i--) {
        int j = A[i] - l;
        S[D[j] - 1] = A[i];
        D[j]--;
    }
    free(D);
}
```

5.3 Örnek: 13 11 12 13 12 12

Farklı değerler 11, 12 ve 13'tür. Aralık 11-13 olduğu için D dizisinin üç gözü vardır: 11 → 0. göz, 12 → 1. göz, 13 → 2. göz.

Sayma. 13'ün gözü bir artar, 11'in gözü bir artar, 12'nin gözü bir artar, 13'ün gözü bir daha, 12'nin gözü bir daha, 12'nin gözü bir daha artar. Frekanslar: 11'den 1, 12'den 3, 13'ten 2 tane.

Birikimli toplam. 1, $1+3=4$, $4+2=6$. Kullanılacak D dizisi `1 4 6` olur.

Yerleştirme. Sondan başa gidilir. Hoca ilk adımda önce "dördüncü göze yaz" dedi, sonra düzeltti: D'deki değerin bir eksiğine yazılır, çünkü indisler 0 ile $n-1$ arasındadır.

İşlenen $A[i]$	D (önce)	Yazılan indis	D (sonra)	Çıktı S
$A[5]=12$	1 4 6	3	1 3 6	_ _ _ 12 _ _
$A[4]=12$	1 3 6	2	1 2 6	_ _ 12 12 _ _
$A[3]=13$	1 2 6	5	1 2 5	_ _ 12 12 _ 13
$A[2]=12$	1 2 5	1	1 1 5	_ 12 12 12 _ 13
$A[1]=11$	1 1 5	0	0 1 5	11 12 12 12 _ 13
$A[0]=13$	0 1 5	4	0 1 4	11 12 12 12 13 13

İlk adımdan sonra D = `1 3 6` olduğunda hoca bunu şöyle okudu: 11 görürsen 0'a, 12 görürsen 2'ye, 13 görürsen 5. indise yaz. Sonuç `11 12 12 12 13 13` olur.

Hoca kafa karışıklığını önlemek için şunu vurguladı: Dersin başında anlatılan counting sort, distribution counting sort olarak da bilinir; ikisi aynı şeydir. Frekanslar sayılır, birikimli hale getirilerek dağılıma çevrilir. Tekil eleman sayısı az olduğunda, yalnızca çıktı için ek bir dizi kullanarak $O(n+k)$ ya da $O(n+r)$ sürede sıralama yapılır.

Hocanın bu bölümden çıkardığı sonuç şuydu: Sıralama her zaman iç içe iki döngü ve değer karşılaştırması olmak zorunda değildir. "Kaç değerden küçüğüm, kaç değer benden büyük, benden kaç tane var, sonuç dizisinde hangi indisler arasında yer alırım?" sorularının cevabı bulunarak da sıralama yapılabilir.

6. Radix Sort: Delikli Kartlardan Gelen Bir Algoritma

Hoca sonra basamak sıralamaya (radix sort) geçti. Bu algoritma tamamen mekanik sıralamadan, yani tarihten gelir. Slaytta IBM'in ürettiği, delikli kartları sıralamak için kullanılan, kolon kolon çalışan bir makine gösterildi. Makinede yan yana dizilmiş bölmeler vardı.

Makine, delikli kartlarda kodlanmış sayıları basamaklarına göre sıralar. Her adımda bir basamağa göre sıralama yapılır. Sonra bir insan kartları toplar, tekrar dizer, makine bir sonraki basamağa göre bir daha sıralar. Basamak sayısı kadar sıralama yapıldığında bütün girdi sıralı hale gelir.

Sıralanan şeylerin sayı olması gerekmez; kelimeler de harf harf sıralanabilir. Radix sort'un her basamakta yaptığı iş, o basamağa göre bir counting sort'tur. Bu sayede karşılaştırma yapmadan, `if` kullanmadan sıralama yapılır.

Basamak basamak sıralama iki yönden yapılabilir:

- **LSD (least significant digit, en düşük anlamlı basamak):** Son haneden başlayıp sola doğru gidilir.
- **MSD (most significant digit, en yüksek anlamlı basamak):** İlk haneden başlayıp sağa doğru gidilir.

7. LSD Radix Sort

7.1 Algoritma

Hoca önce daha basit olan LSD radix sort'u anlattı. Girdi n tane dizgidir (string), örneğin n tane kelime. Bir alfabe vardır; slaytta 0'dan σ'ya kadar semboller diye geçer. Sayılar sıralansaydı alfabe 0-9 rakamlarından oluşan 10 sembollü bir alfabe olurdu. Her dizginin uzunluğu sabittir. Hoca algoritmada bu uzunluğa m, karşılaştırma tablosunda W dedi; bu notta W kullanılıyor. Çıktı, dizgilerin artan sırada dizilmiş halidir.

Algoritma basamak sayısı kadar dönen tek bir döngüdür: $d = W - 1$'den 0'a kadar, girdiyi d. karaktere göre kararlı bir counting sort ile sırala. Yapılan tek şey budur.

```
LSD-RADIX-SORT(A, W)
  her d = W-1..0 için
    A'yı d. karaktere göre kararlı counting sort ile sırala
```

> **Ders sonrası not:** Hoca karmaşıklığı önce "basamak sayısı artı σ", sonra "$n+k$ gibi" diye söyledi. Her basamak için bir counting sort yapıldığından ve her counting sort $O(n + R)$ sürdüğünden, LSD'nin karmaşıklığı $O(W(n + R))$'dir. Burada R alfabe boyutudur (slayttaki σ).

C karşılığı (derste C kodu yazılmadı; `gcc -Wall` ile denendi). Her geçişte sayaç dizisi `count` ile kararlı bir dağıtım yapılır:

```c
#include <stdlib.h>

#define R 256   /* alfabe boyutu (ASCII) */

/* n tane, her biri W karakterli dizgi */
void lsd_sort(char *a[], int n, int W)
{
    char **aux = malloc(n * sizeof(char *));
    for (int d = W - 1; d >= 0; d--) {
        int count[R + 1] = {0};
        for (int i = 0; i < n; i++)
            count[(unsigned char)a[i][d] + 1]++;
        for (int r = 0; r < R; r++)
            count[r + 1] += count[r];
        for (int i = 0; i < n; i++)           /* kararlı dağıtım */
            aux[count[(unsigned char)a[i][d]]++] = a[i];
        for (int i = 0; i < n; i++)
            a[i] = aux[i];
    }
    free(aux);
}
```

7.2 Örnek 1: Üç Harfli Dört Kelime

Slaytta dört tane üç harfli kelime vardı: CAT, HIM, HAM, BAT. Hoca bunların sayı da olabileceğini söyleyip her harfe bir rakam verdi: CAT = 318, HIM = 457, HAM = 417, BAT = 218.

Kelimeler alt alta yazılır ve en düşük anlamlı basamaktan, yani son harften başlanır.

3. harfe göre. Son harfler T, M, M, T'dir. M, T'den küçük olduğu için iki M yukarı çıkar. HIM girdide HAM'dan önce geldiği için yukarıda kalır. Sıra: HIM, HAM, CAT, BAT. Bu sıranın korunması counting sort'un sırayla işlemesinden gelir; girdideki sıra her basamakta mümkün olduğunca korunur.

Hoca burada sordu: Bunlar şu anda sıralı mı? Hayır. Ama en düşük anlamlı basamaktaki değerler kendi içinde sıralandı: M, M, T, T.

2. harfe göre. İkinci harfler I, A, A, A'dır. A'lar yukarı çıkar, sıraları korunur: önce HAM, sonra CAT, sonra BAT. I en sona gider. Sıra: HAM, CAT, BAT, HIM. Hoca, "bu sıra bozuldu hocam" itirazına "Bozulsun, önemi yok" diye cevap verdi. Önemli olan her basamağın kendi içinde sıralanması ve en yüksek anlamlı basamağa doğru ilerlemektir.

1. harfe göre. İlk harfler H, C, B, H'dir. B alfabede A'ya en yakın olduğu için en üste çıkar, sonra C gelir. İki H kelimesi, HAM ve HIM, aynı ilk harfe sahiptir. Bir önceki adımda A, I'dan önce sıralandığı için HAM yukarıda, HIM aşağıda kalır. Daha düşük anlamlı basamak daha önce sıralandığı için bu kelimeler kendi içinde sıralı kalmaya devam eder.

Adım	Sıra
Başlangıç	CAT, HIM, HAM, BAT
3. harfe göre	HIM, HAM, CAT, BAT
2. harfe göre	HAM, CAT, BAT, HIM
1. harfe göre	BAT, CAT, HAM, HIM

7.3 Örnek 2: Aynı Örnek Sayılarla

Aynı sıralama `318 457 417 218` sayılarıyla yapılır.

Adım	Bakılan basamaklar	Sıra
Başlangıç		318, 457, 417, 218
Birler basamağı	7, 7, 8, 8	457, 417, 318, 218
Onlar basamağı	1, 1, 1, 5	417, 318, 218, 457
Yüzler basamağı	2, 3, 4, 4	218, 318, 417, 457

Yüzler basamağında 417 ve 457'nin ikisi de 4 ile başlar. 417 bir önceki adımda daha yukarıda olduğu için çıktıda da önce yer alır. Sonuç `218 318 417 457` olur. Hocanın dediği gibi, rakamlar da sonuçta birer semboldür.

7.4 Örnek 3: On İki Kelime

Slaytta daha büyük bir örnek vardı: üç harfli on iki kelime. Hoca önce son harfe göre sıraladı: B'ler, D'ler, E'ler. A'dan Z'ye doğru dizildiler ve sıra korundu. Slayttaki oklar birbirini kesmiyordu: Girdide önce gelen yukarıda, sonra gelen aşağıda kalıyordu. Örneğin girdideki en üstteki D, sıralanmış D'lerin içinde de en üstte; en alttaki D yine en altta. Sonra ikinci harfe göre, en son da en anlamlı harfe göre sıralandı.

Adım	Sıra
Başlangıç	dab, add, cab, fad, fee, bad, dad, bee, fed, bed, ebb, ace
3. harfe göre	dab, cab, ebb, add, fad, bad, dad, fed, bed, fee, bee, ace
2. harfe göre	dab, cab, fad, bad, dad, ebb, ace, add, fed, bed, fee, bee
1. harfe göre	ace, add, bad, bed, bee, cab, dab, dad, ebb, fad, fed, fee

Hoca, son adımın neden doğru çalıştığını F'li kelimeler üzerinden gösterdi. F ile başlayan fad, fed, fee bir önceki adımda ikinci harfe göre A, E, E diye sıralanmıştı. fed ve fee'nin ikinci harfi aynı (E) olduğu için bunlar daha önce üçüncü harfe göre D, E diye sıralanmıştı. Her basamak kendi içinde sıralanarak geldiği için sonunda bütün elemanlar sıralı olur.

> **Ders sonrası not:** Hoca ikinci harfe göre sıralamada harfleri "A, A, A, A, A, B, C, D, E, E, E" diye saydı. Bu kelimelerde ikinci harfi E olan dört kelime vardır (fed, bed, fee, bee); doğrusu A×5, B, C, D, E×4'tür.

7.5 Tarihçe

Bu yöntem, mekanik sıralamaya çok uygundur ve Herman Hollerith tarafından icat edilmiştir. Hoca, bu makinenin ABD nüfus sayımında on yıldan fazla sürecek sonuçları birkaç ay içinde açıklayarak rekor kırdığını ve IBM'in temellerini attığını söyledi.

> **Ders sonrası not:** Hollerith'in makinesi büyük olasılıkla 1890 ABD nüfus sayımında kullanıldı. Hollerith'in kurduğu şirket de büyük olasılıkla daha sonra IBM'i oluşturan birleşmeye katıldı. Slayttaki makineyi hoca "IBM'in ürettiği" diye tanıttı; bu tür kart sıralama makineleri büyük olasılıkla sonradan IBM tarafından üretildi.

7.6 LSD Nerede Kullanılır?

Hocaya göre LSD, sabit uzunluklu anahtarlardan oluşan çok büyük tablolar için biçilmiş kaftandır. Örnekler:

- Banka hesap numaraları, IBAN'lar. IBAN sabit uzunluktadır, ülke koduyla (TR) başlar, gerisi rakamlardır.
- T.C. kimlik numaraları: 11 haneli sayılar.
- GUID (globally unique identifier) gibi benzersiz tanımlayıcı anahtarlar.
- Veritabanı indekslerinin oluşturulması.

Hocanın sayısal örneği şuydu: 85 milyon kimlik numarasını sıralamak istiyoruz. Karşılaştırmalı bir algoritma yaklaşık $n \log n$ adım ister. Insertion sort mu, merge sort mu, quicksort mu, heap sort mu, yoksa LSD mi kullanmalıyız? LSD ile yalnızca 11 tane counting sort yapmak yeter. Alfabe ASCII ise 256, Unicode ise 65536 boyutlu bir sayaç dizisi yeterlidir, çünkü o kadar farklı karakter vardır. Sabit uzunluklu dizgiler W adımda sıralanır.

Hesap (ara adımlar eklendi):

- Karşılaştırmalı sıralama: $n \log_2 n = 85 \times 10^6 \times \log_2(85 \times 10^6) \approx 85 \times 10^6 \times 26{,}3 \approx 2{,}24$ milyar adım.
- LSD: 11×85 milyon $= 935$ milyon adım.

  Ders sonrası not: Hoca 11×85 milyon için "850 milyon adım" dedi. Doğrusu 935 milyondur. Sonuç değişmez: LSD doğrusal zamanda, karşılaştırmalı sıralamadan belirgin biçimde az adımla biter.

Hocaya göre bu tür problemlerde, özellikle sembol kümesi küçükse, kimse merge sort ya da quicksort ile uğraşmaz; LSD radix sort kullanılır.

8. MSD Radix Sort

8.1 Fikir: Soldan Başlamak

Hoca sonra şunu sordu: En düşük anlamlı basamaktan başlamak yerine, insanın okuduğu gibi soldan, yani en yüksek anlamlı basamaktan başlayarak sıralama yapamaz mıyız? Yapabiliriz. Bu durumda algoritma quicksort'a çok benzer biçimde çalışır.

Quicksort böl ve yönet (divide and conquer) yaklaşımıyla diziyi üç parçaya böler: bir pivot, pivottan küçükler ve pivottan büyükler. Hoca quicksort'un ayrıntısının sonraki yıl görüleceğini söyledi. MSD ise diziyi alfabe kadar parçaya böler. Aradaki temel fark budur. Her adımda, o adımda sıralanan karakter konumundaki farklı karakter sayısı kadar parça oluşur. Her parça kendi içinde özyinelemeli (recursive) olarak tekrar MSD'ye gönderilir. Gerekirse son karaktere kadar devam edilir.

8.2 Örnek: Sözlükteki Kelimeler

Slayttaki ilk örnek bir sözlük parçasıydı: alphabet, alignment, allocate, algorithm, alternative, alias, alternate, all. Bu kelimelerin hepsi "al" ile başlar. Birinci ve ikinci en anlamlı basamaklar sıralanınca tek bir grup kalır ve üçüncü harfe gelinir.

Üçüncü harfler P, I, L, G, T, I, T, L'dir. Yani G, I, L, P ve T olmak üzere beş farklı harf ve beş alt küme oluşur. Hoca bu noktada önemli bir avantajı vurguladı: Her alt küme, sanki öbürleri yokmuş gibi bağımsız olarak MSD'ye verilir. Bu yüzden parçalar paralel çalıştırılabilir. Biri bir işlemcide, öbürü başka bir işlemcide sıralanabilir.

- **G grubu:** Yalnız algorithm vardır. Grup bitti; kelimenin geri kalanına hiç bakılmaz.
- **I grubu:** alignment ve alias. Dördüncü harfler G ve A'dır. A yukarıda, G aşağıda kalır: alias, alignment. Geri kalan harflere bakmaya gerek kalmaz.
- **L grubu:** allocate ve all. all'dan sonra kelime biter. Kelimenin sonu (null karakter, `\0`) bütün harflerden önce gelir. Bu yüzden all yukarı çıkar, allocate aşağı iner. all bittiği için allocate'in devamına bakmaya gerek kalmaz.
- **P grubu:** Yalnız alphabet vardır; kontrol bile gerekmez.

- **T grubu:** alternative ve alternate. E, R, N, A, T harflerinin hepsine tek tek bakılır. Hoca burada “patladım” dedi: Bu iki kelimede neredeyse sona kadar her karakteri kontrol etmek ve MSD’yi çağırmaya devam etmek gerekir.

Sıralı sonuç: algorithm, alias, alignment, all, allocate, alphabet, alternate, alternative.

8.3 Algoritma

Girdi $R = \{S_1, \ldots, S_n\}$ dizgi kümesidir; alfabe yine 0 ile σ arasındadır. LSD’den farkı, girdilerin aynı uzunlukta olmasının gerekmemesidir. Soldan sağa gidildiği için değişken uzunluklu dizgilerle çalışılabilir; bir dizgi bittiğinde (`\0`) o dizgi için iş biter.

Slayttaki algoritmada önemli bir iyileştirme vardı: Parçanın boyutu alfabe boyutu σ’dan küçükse MSD ile hiç uğraşılmaz, doğrudan quicksort yapılır.

```
MSD-RADIX-SORT(R, d)
  eğer |R| < σ ise
    R'yi quicksort (ya da insertion sort) ile sırala ve dön
  R'yi d. karaktere göre counting sort ile gruplara ayır
    (dizgisi bitenler en başa)
  her grup R_c için
    MSD-RADIX-SORT(R_c, d+1)
```

C karşılığı (derste C kodu yazılmadı; `gcc -Wall` ile denendi). Küçük parçalar için insertion sort kullanıldı; hoca quicksort ya da insertion sort çağrılmasının fark etmeyeceğini söyledi. Dizgi sonu `-1` ile gösterilir, böylece her harften küçük olur:

```c
#include <stdlib.h>
#include <string.h>

#define R 256       /* alfabe boyutu (ASCII) */
#define CUTOFF 15   /* bu boyuttan küçük parçalar başka algoritmayla sıralanır */

static int char_at(const char *s, int d)
{
    return d < (int)strlen(s) ? (unsigned char)s[d] : -1;   /* dizgi sonu: -1 */
}

static void insertion(char *a[], int lo, int hi, int d)
{
    for (int i = lo; i <= hi; i++)
        for (int j = i; j > lo && strcmp(a[j] + d, a[j - 1] + d) < 0; j--) {
            char *t = a[j]; a[j] = a[j - 1]; a[j - 1] = t;
        }
}

static void msd(char *a[], char *aux[], int lo, int hi, int d)
{
    if (hi <= lo + CUTOFF) {                  /* küçük parça */
        insertion(a, lo, hi, d);
        return;
    }
    int count[R + 2] = {0};
    for (int i = lo; i <= hi; i++)            /* sayma */
        count[char_at(a[i], d) + 2]++;
    for (int r = 0; r < R + 1; r++)           /* birikimli toplam */
        count[r + 1] += count[r];
    for (int i = lo; i <= hi; i++)            /* kararlı dağıtım */
        aux[count[char_at(a[i], d) + 1]++] = a[i];
    for (int i = lo; i <= hi; i++)
        a[i] = aux[i - lo];
    for (int r = 0; r < R; r++)               /* her harf grubu için özyineleme */
```

```
        msd(a, aux, lo + count[r], lo + count[r + 1] - 1, d + 1);
}

void msd_sort(char *a[], int n)
{
    char **aux = malloc(n * sizeof(char *));
    msd(a, aux, 0, n - 1, 0);
    free(aux);
}
```

8.4 Örnek: On İki Kelime

Hoca LSD'deki on iki kelimelik örneği bu kez MSD ile sıraladı. Başlangıç ilk harftir. İlk harfler A, B, C, D, E ve F'dir. Counting sort yapılınca her harfin başlangıç indisi bulunur:

İlk harf	A	B	C	D	E	F
Kelime sayısı	2	3	1	2	1	3
Başlangıç indisi	0	2	5	6	8	9

Örneğin A'lar (add ve ace) 0 ve 1. indislerde, B'ler 2. indisten başlayarak yer alır. Böylece elimizde altı tane ayrık problem oluşur. Soldaki basamak counting sort ile sıralandı; artık her grubun içine özyinelemeli olarak girilip yine counting sort yapılır. Gruplar birbirinden bağımsız olduğu için paralel çalıştırılabilir.

> **Ders sonrası not:** Hoca grupları sayarken "2 tanesi D, 2 tanesi E, 3 tane de F" dedi. E ile başlayan yalnız bir kelime vardır (ebb). Başlangıç indisleri de bunu gösterir: E 8'den, F 9'dan başlar.

Grupların içi:

- **A grubu (add, ace):** İkinci harfler D ve C'dir. Counting sort ile C sıfırıncı, D birinci olur: ace, add. Her alt grupta tek eleman kaldığı için üçüncü harflere bakılmaz.
- **B grubu (bad, bed, bee):** İkinci harfler A, E, E'dir. A tek elemandır, kendi içinde sıralıdır. E grubunda iki kelime vardır; özyinelemeyle üçüncü harflere (D ve E) bakılır: bed yukarı, bee aşağı.
- **C grubu (cab):** Tek eleman vardır. A ve B harflerine hiç dokunulmaz.
- **D grubu (dab, dad):** İkinci harf ikisinde de A'dır. Üçüncü harfler B ve D'dir: dab yukarı, dad aşağı. Zaten bu sıradaydılar.
- **E grubu (ebb):** Tek eleman; hiçbir şey yapmadan sıralıdır.
- **F grubu (fad, fed, fee):** İkinci harfler A, E, E'dir. A tek elemandır. E grubunda üçüncü harfler D ve E'dir: fed yukarı, fee aşağı.

Sonuç: ace, add, bad, bed, bee, cab, dab, dad, ebb, fad, fed, fee. Bu örnekte hepsi aynı uzunlukta kelimeler kullanıldı, ama MSD için bu zorunlu değildir.

8.5 Örnek: "She Sells Seashells"

Hoca kitaptan değişken uzunluklu bir örnek verdi. Kaynak, altyazıda "Sevgi Ebit'in kitabı" diye geçen kitaptır.

> **Ders sonrası not:** Altyazıda "Sevgi Ebit" diye geçen ad büyük olasılıkla Robert Sedgewick ve Kevin Wayne'in *Algorithms* kitabıdır. Bu örnek ve on iki kelimelik örnek o kitapta yer alır.

Girdi bir İngilizce tekerlemedir: she sells seashells by the sea shore the shells she sells are surely seashells.

1. harf. İlk harfler A, B, S ve T'dir. Önce A, sonra B, sonra S'ler, en son T'ler gelir. A grubunda tek kelime (are), B grubunda tek kelime (by) vardır. Bunların geri kalan harflerine bakmaya gerek yoktur; iki eleman hemen sıralanmış olur. S'li çok kelime vardır, onlar için özyineleme gerekir.

S grubu, 2. harf. İkinci harfler E, H ve U'dur. Counting sort ile sıralanınca girdideki geliş sırası korunur, oklar kesişmez:

- E'liler (5 kelime): sells, seashells, sea, sells, seashells.
- H'liler (4 kelime): she, shore, shells, she.
- U'lu (1 kelime): surely. Tek eleman olduğu için içine girilmez.

SE grubu, 3. harf. Üçüncü harfler L, A, A, L, A'dır; yalnız A ve L vardır. A'lar yukarı, L'ler aşağı gider, geliş sırası korunur: seashells, sea, seashells, sells, sells.

- **SEA grubu:** sea kelimesi burada biter. Dizgi sonu (null karakter) en küçük olduğu için sea yukarı gider. Kalan iki seashells için S, H, E, L, L, S karakterlerine tek tek bakılır. Hoca bunu şöyle açıkladı: Son karaktere kadar hepsi aynı olsa bile tek tek karşılaştırmak zorundayız; son karakter farklı olsaydı bunu bilemezdik.
- **SEL grubu:** sells ve sells için de L'ye ve S'ye bakılır; sıra değişmez.

SH grubu, 3. harf. Üçüncü harfler E, O, E, E'dir (she, shore, shells, she). E'liler (she, shells, she) yukarı, shore aşağı gider. E'lilerin içinde iki she dördüncü karakterde biter ve yukarı çıkar; shells en sona kalır. shore tek eleman olduğu için kendi içinde sıralıdır. Sıra: she, she, shells, shore.

T grubu. İki "the" vardır. T'ye, H'ye, E'ye bakılır; ikisi aynıdır ve sıralıdır.

Sonuç: are, by, sea, seashells, seashells, sells, sells, she, she, shells, shore, surely, the, the.

8.6 Kötü Yanları

Hoca önce MSD'nin kötü yanlarını anlattı:

- **Küçük parçalarda yavaştır.** Parçalar çok küçükse sürekli özyinelemeli çağrı yapılır. Az elemanlı parçalar için bile her seferinde sayaç dizisi sıfırlanır ve sayma yapılır. Algoritmadaki "boyut küçükse quicksort yap geç" adımı bu yüzden vardır. Bir yerden sonra quicksort ya da insertion sort gibi bir algoritmayı çağırmak daha mantıklıdır.
- **En kötü durum.** Bütün harfler farklıysa çok fazla parça oluşur. Çok fazla özyinelemeli dallanma ve geri toplama yapılır, radix sort birden çok yavaşlar. Çok küçük parçalar oluşacaksa MSD radix sort kullanmak mantıklı değildir.

8.7 İyi Yanları

MSD'nin iyi yanı, değişken uzunluklu girdilerde, özellikle sözlük tarzı uygulamalarda çok iyi sonuç vermesidir. Karakterlerin büyük kısmına hiç bakmadan sıralama yapılabilir.

Hoca bunu önce on iki kelimelik sabit uzunluklu örnekte gösterdi. A grubunda C ve D'ye bakmak yeter; ace ve add'in son harflerine bakılmaz. bad tek kaldığı için son harfine bakılmaz. bed ve bee'de son harflere bakmak zorunludur. cab'ın ikinci ve üçüncü harflerine hiç bakılmaz. D grubunda ise son harfe kadar gidilir. Hoca bu örnekte 24 karakterin 5'ine bakılmadığını, 19'una bakıldığını, yani karakterlerin yaklaşık %80'ine bakıldığını söyledi.

> **Ders sonrası not:** Örnekte 8 değil 12 kelime vardır; toplam $3 \times 12 = 36$ karakter eder. Tek elemanlı gruplara girilmediği varsayılarak sayılınca (ara adım eklendi): 1. harfte 12, 2. harfte 10 (C ve E grupları tek elemanlı), 3. harfte 6 karaktere (bed, bee, dab, dad, fed, fee) bakılır. Toplam $12 + 10 + 6 = 28$ karakterdir; 8 karaktere hiç bakılmaz. Oran $28/36 \approx \%78$'dir, hocanın verdiği yaklaşık %80'e yakındır.

Değişken uzunluklu sözlük örneğinde kazanç daha büyüktü. Slayttaki kelimelerde C ile başlayan tek kelime olduğu için onun geri kalan karakterlerine hiç bakılmaz. A ile başlayan iki kelimede (hoca "aseton" ve "addition" diye okudu) yalnız ikinci harflere, C ve D'ye bakmak yeter. B grubundaki bedazzled ve beehive için D ve E'ye bakmak yeter. Hoca toplam 64 kadar karakterden yalnızca 19'una bakıldığını, yani karakterlerin yaklaşık %30'unun sıralamaya girdiğini, buna rağmen bütün girdinin sıralandığını söyledi.

Hocanın özeti şuydu: Girdi uygunsa MSD radix sort çok fayda sağlar, uygun değilse çok zarar da verebilir. Sabit boyutlu girdiler varsa, karakter kümesi çok geniş değilse ve tekrar sayısı fazlaysa (kimlik numarası gibi) LSD radix sort milyonlarca, milyarlarca kaydı doğrusal zamanda sıralamayı sağlar. Bunlar gündelik hayatta kullanılan sıralama yöntemleridir.

9. Sıralamanın Tanımı ve Önemi

Hoca son bölüme şu soruyla geçti: Sıralama için klasik karşılaştırma temelli yöntemler, bölerek sıralayan yöntemler (merge sort, quicksort), hiç karşılaştırma yapmayan ya da basamaklar üzerinde çalışan yöntemler var. Bunları nasıl karşılaştıracağız?

Önce sıralamanın tanımını tekrar verdi. Sıralama, aynı türden nesnelerden oluşan bir koleksiyonu mantıksal bir düzene sokmaktır. Bu düzen büyüklük-küçüklük ilişkisidir. $\langle a_1, a_2, \dots, a_n \rangle$ biçiminde verilen bir girdi, öyle bir $\langle a'_1, a'_2, \dots, a'_n \rangle$ dizisine dönüştürülür ki her eleman bir öncekinden büyük ya da ona eşittir:

$$a'_1 \leq a'_2 \leq \dots \leq a'_n$$

Sıralama çok sayıda algoritmanın içinde doğal olarak geçer: işlemlerin işlenmesi, optimizasyon problemleri, astrofizik, çeşitli dinamik çözümlemeler, moleküler biyoloji, dil bilimi, gen bilimi, hava tahmini. Birçok algoritmanın ilk adımı sıralamadır. Hoca, 20. yüzyılın en iyi 10 algoritması oylandığında bunlardan birinin bir sıralama algoritması, quicksort olduğunu söyledi. Sıralama bu kadar önemlidir ve hâlâ üzerinde çalışılan bir araştırma alanıdır.

10. Sıralama Algoritmalarını Karşılaştırma Ölçütleri

Bir problemde sıralama yapacağız ve hangi algoritmayı kullanacağımıza karar vermemiz gerekiyor. Hocaya göre temelde üç ölçüt vardır.

10.1 Performans

Performans (başarım), öncelikle çalışma zamanı karmaşıklığıdır: Algoritma ne kadar sürüyor, büyük O ya da büyük Θ gösterimindeki karmaşıklığı nedir? Performans, karşılaştırma ve yer değiştirme (swap) sayısıyla ölçülebilir. Counting sort'taki gibi yer değiştirme yoksa toplam dizi erişimi (bellek erişimi) sayısı sayılabilir.

10.2 Ek Bellek: Yerinde ve Yerinde Olmayan Sıralama

İkinci ölçüt mekân (uzay) karmaşıklığıdır; buna ekstra bellek kullanımı da denir.

- **Yerinde sıralama (in-place sort):** Hocanın benzetmesi bir kitap rafıydı. Raftaki kitapları raf üzerinde sıralıyorsunuz. Bir kitabı çıkarıp başka birinin yerine koyuyor, ikisini çıkarıp birini kaydırıyorsunuz. Yalnız elinizi ya da iki elinizi kullanabilirsiniz, başka hiçbir yer yok. Algoritma için bunun karşılığı sabit sayıda ek değişken ya da küçük bir fonksiyon çağrı yığınıdır (function call stack).
- **Yerinde olmayan sıralama (out-of-place sort):** "Yerim bol" deyip bütün kitapları raftan çıkarıp yere döküyor, sonra sıralayarak geri diziyorsunuz. Ya da bir raftakileri sıralayarak başka bir rafa yerleştiriyorsunuz; iki raf gerekir. Çıktı için harici bir bellek alanı kullanan algoritmalar böyledir. Counting sort ve merge sort örnektir.

10.3 Kararlılık

Üçüncü ölçüt, tekrar eden değerlerin nasıl işlendiğidir: Girdideki sıra korunuyor mu? Korunuyorsa algoritmaya kararlı ya da istikrarlı (stable) sıralama denir. Girdide tekrar eden değerler çıktıda da aynı sırayla yer alır. Counting sort'ta önde olan eleman yukarıda, sonra gelen aşağıda kalıyordu; demek ki girdideki sırayı korur.

Hocanın örneği yaşa göre sıralanan öğrencilerdi. 5 yaşındaki Ayşe sıraya daha önce girmiş, Veli daha sonra gelmiş olsun. Kararlı bir sıralamada Ayşe sıralamadan sonra da önde, Veli arkada kalır. Algoritma bunun garantisini veremiyorsa, yani yaşa göre sıralarken Veli Ayşe'nin önüne geçebiliyorsa, buna kararsız ya da istikrarsız (unstable) sıralama denir.

Özetle sıralama algoritmaları üç temel ölçütle değerlendirilir: performans, ekstra bellek gereksinimi ve tekrar eden değerlerin nasıl işlendiği.

11. Tasarım Yaklaşımlarına Göre Sıralama Algoritmaları

Hoca sıralama algoritmalarını algoritma tasarım yaklaşımlarına göre gruplandırdı.

- **Kaba kuvvet (brute force):** Selection sort, bubble sort ve hocaya göre shell sort. Selection sort ve bubble sort'ta körlemesine karşılaştırılır; büyükse ya da küçükse yer değiştirilir.
- **Azalt ve yönet (decrease and conquer):** Girdi uzayını azaltarak ilerlenir. Insertion sort bu yaklaşımın örneğidir.
- **Dönüştür ve yönet (transform and conquer):** Girdi başka bir veri yapısına dönüştürülür ve o yapının özelliği kullanılarak çözüm geliştirilir. Heap sort'ta girdi bir heap organizasyonuna çevrilir; n tane $\log n$'lik heapify işlemiyle problem çözülür.
- **Böl ve yönet (divide and conquer):** Girdi bir mantığa göre küçük parçalara ayrılır, parçalar daha küçük problemler olarak sıralanır, sonra birleştirilerek (combine) çıktı oluşturulur. Merge sort ve quicksort bu yaklaşımdadır. Hoca bunların bu ders kapsamında görülmediğini, algoritma analizi dersine bırakıldığını söyledi.

  Ders sonrası not: Hoca shell sort'u kaba kuvvet algoritmaları arasında saydı. Shell sort, aralıklı (gap) insertion sort uygulamasıdır ve büyük olasılıkla kaynaklarda insertion sort'un bir genelleştirmesi olarak, yani azalt ve yönet ailesine yakın olarak ele alınır.

11.1 Merge Sort

Hoca ayrıntıya girmeden fikri anlattı. Elimizde iki sıralı dizi varsa bunları düzgün biçimde birleştirerek sıralı bir dizi elde edebiliriz. O halde dizi ortadan ikiye bölünür: $n/2$ boyutlu iki problem oluşur. Bunlar da ikiye bölünür: $n/4$ boyutlu dört problem. Sonra $n/8$ boyutlu sekiz problem. Dizi 8 elemanlıysa artık her biri tek elemanlı 8 küçük dizi vardır.

Tek elemanlı bir dizi tanım gereği sıralıdır, çünkü büyüklük-küçüklük ilişkisi kurulacak başka eleman yoktur. Yani bölme (divide) yapıldı, çözme (conquer) adımında hiçbir şey yapılmadı. Sonra parçalar birleştirilir: ikişer tek elemanlı parça birleştirilip ikililer, ikililer birleştirilip dörtlüler, dörtlüler birleştirilip sekizli yapılır.

Merge sort'un artısı $O(n \log n)$ karmaşıklıkla sıralamasıdır. $\log n$, ikiye bölme ve ikişer birleştirme mantığından gelir. Eksisi, birleştirme sırasında çıktıyı oluşturmak için ek bir diziye ihtiyaç duymasıdır. Bu yüzden n ile orantılı ek alan gerekir.

11.2 Quicksort

Quicksort çok popüler ve çok bilinen bir algoritmadır. Genelde $O(n \log n)$'de çalışır, en kötü durumda $O(n^2)$'ye çıkabilir; hoca bunun çok özel bir durum olduğunu söyledi. Merge sort diziyi ortadan ikiye bölerken, quicksort bölmeyi değerlere göre yapar. Bir değer, yani pivot seçilir. Pivottan küçükler bir tarafa, büyükler bir tarafa ayrılır. Her parça için tekrar bir pivot seçilip aynı işlem yapılır. Bölerek ilerlendiği için $\log n$ mertebesi gelir; n eleman üzerinde $\log n$ derinlikte işlem yapıldığından kabaca $n \log n$'de çalışır. Ayrıca $c \log n$'lik bir ek alan ihtiyacı vardır. Hoca ayrıntıya girmedi.

11.3 Radix Sort

Radix sort'ta basamak basamak sıralama yapılır. Gerektiğinde karşılaştırma yapmadan, hatta mekanik bir cihazla bile sıralama yapılabilir. En düşük anlamlı basamaktan başlanırsa LSD, en yüksek anlamlı basamaktan başlanırsa MSD radix sort elde edilir. MSD özellikle değişken boyutlu girdilerde kullanılır.

12. Karşılaştırma Tablosu ve Algoritma Seçimi

Hoca bütün bu algoritmaları bir tabloda topladı. Karmaşıklık sütununda en iyi ve en kötü durumları ayırmadığını, kabaca ortalama değerleri verdiğini söyledi.

Algoritma	Kararlı	Yerinde	Zaman (yaklaşık)	Ek alan
Selection sort	Hayır	Evet	n^2 mertebesi	1
Insertion sort	Evet	Evet	$n^2/2$	1
Shell sort	Hayır	Evet	$n^{3/2}$	1
Merge sort	Evet	Hayır	$n \lg n$	n
Quicksort	Hayır	Evet	$1{,}39\, n \lg n$	$c \lg n$
Heap sort	Hayır	Evet	$2n \lg n$	1
LSD radix sort	Evet	Hayır	$2nW$	$n + R$
MSD radix sort	Evet	Hayır	$2nW$	$n + DR$

Hocanın tablo üzerindeki açıklamaları:

- **Karmaşıklık:** Insertion sort n^2 mertebesindedir, ama tam olarak $n^2/2$'dir. Shell sort da n^2 mertebesinde sayılır, ama aslında $n^{3/2}$'dir. Quicksort tam olarak $1{,}39\, n \lg n$, heap sort $2n \lg n$'dir. Radix sort doğrusal, $O(n)$ görünür, ama aslında $2nW$'dir. Bu nW çarpanını unutmamak gerekir.
- **Büyük alfabe:** Alfabe çok büyükse MSD ya da LSD kullanmak anlamsızlaşır. Küçük alfabe ve çok tekrar varsa bu algoritmalar büyük kazanç sağlar.
- **Kararlılık:** Selection sort girdideki sırayı gözetmez; sıranın korunması gerekiyorsa kullanılamaz. Insertion sort gözetir. Merge sort gözetir, quicksort gözetmez. Radix sort'larda yukarıdaki yukarıda, aşağıdaki aşağıda kalır, oklar kesişmez; bu yüzden kararlıdır. Heap sort, heap organizasyonu sırayı değiştirebildiği için girdi sırasını korumaz.
- **Yerinde sıralama:** Selection, insertion, shell ve bubble sort yerinde sıralar; tek bir `temp` değişkeni yeter. Merge sort ve radix sort yerinde sıralayamaz; n boyutlu ek bir çıktı dizisi gerekir. Radix'te buna sayaç dizisi eklenir: LSD'de $n + R$. MSD'de özyinelemeli çağrılar olduğu için her çağrı derinliğinde ayrı bir sayaç dizisi gerekir; ek alan $n + D \cdot R$ olur (D: çağrı yığınının derinliği). Quicksort yerinde sıralar ama $c \log n$ ek alana ihtiyaç duyar. Heap sort yerinde sıralar, ek alan gerekmez; heapify aynı dizi üzerinde tekrar tekrar yapılır.

  Ders sonrası not: Hoca bubble sort'un da sırayı gözetmediğini söyledi. Yalnız $A[j] > A[j+1]$ olduğunda yer değiştiren standart bubble sort kararlıdır; eşit elemanlar hiç yer değiştirmez. Tablodaki kararsız algoritmalar selection sort, shell sort, quicksort ve heap sort'tur.

  Ders sonrası not: Hoca "merge sort gözetir, quicksort gözetmez" dedikten sonra "değere bağlı olarak bölükleme yaptığı için, bu konuma bağlı olarak yaptığı için gözetiyor" diye ekledi. Doğru ayrım şudur: Merge sort diziyi konuma göre böler ve birleştirmede eşitlikte soldakini önce alırsa kararlıdır. Quicksort değere (pivota) göre böler ve birbirinden uzak elemanları yer değiştirir; bu yüzden kararlı değildir.

  Ders sonrası not: Hoca $2nW$'deki W'nin "ya basamak sayısı ya da alfabe boyutu" olduğunu söyledi. W anahtar uzunluğudur (basamak ya da karakter sayısı). Alfabe boyutu R'dir ve ek alana ($n + R$) ve her geçişteki sayaç işlemine girer.

Hocanın son mesajı şuydu: Algoritma seçimi bir ödünleşimdir (trade-off). Kararlılık gerekiyor mu? Yerinde sıralama zorunlu mu, yoksa ek bir dizi kullanılabilir mi? Karmaşıklık tatmin edici mi? Bu sorulara göre bir seçim yapılır ve neden o algoritmanın seçildiğine karar verilerek sıralama algoritması çözümün bütününde kullanılır. Tabloya çok daha fazla sıralama algoritması da eklenebilir; derste yalnız konuşulanlar eklendi.

Ek bilgi (derste anlatılmadı): Tablodaki karmaşıklık ve ek alan değerleri, Sedgewick ve Wayne'in *Algorithms* kitabındaki sıralama karşılaştırma tablosuyla büyük olasılıkla aynıdır. O tabloda radix sort satırlarındaki $2nW$ değeri karakter erişim sayısını gösterir, karşılaştırma sayısını değil. Bu yüzden radix sort'un satırı, karşılaştırma sayan öbür satırlarla birebir aynı birimde değildir.

Örnek Sorular

Bu derste ekranda ayrı bir soru ya da alıştırma gösterilmedi. Aşağıdaki sorular derste sözlü olarak soruldu.

1. (derste sözlü, 00:10:03) Counting sort'ta `if` kullanılmadığı söyleniyor. Ama `for` döngüsünün kendisi bir kontrol, yani bir `if` içermiyor mu?

> **Çözüm:** Hoca, döngünün bitiş koşulunun kontrol edildiğini kabul etti, ama bunu döngünün parçası saydıklarını, ayrıca `if` olarak değerlendirmediklerini söyledi. Counting sort'un hiçbir döngüsünde elemanlar birbiriyle karşılaştırılmaz. Adım sayısı baştan belli olduğu için mimari döngüyü farklı biçimde de kurabilir.

2. (derste sözlü, 00:12:22) Counting sort'ta bir elemandan küçük 10 tane eleman varsa bu ne demektir?

> **Çözüm:** O eleman sıralı dizide 11. gözde durmalıdır. Dizi 0'dan başladığı için bu, indis 10'dur.

3. (derste sözlü, 00:38:12) LSD radix sort'ta CAT, HIM, HAM, BAT son harfe göre sıralanınca HIM, HAM, CAT, BAT elde edildi. Bu kelimeler şu anda sıralı mı?

> **Çözüm:** Hayır. Yalnızca en düşük anlamlı basamaktaki harfler kendi içinde sıralıdır (M, M, T, T). Kelimelerin tamamı ancak bütün basamaklar sağdan sola işlendikten sonra sıralı olur.

4. (derste sözlü, 00:44:39) 85 milyon T.C. kimlik numarası (11 haneli) sıralanacak. Insertion sort mu, merge sort ya da quicksort mu, heap sort mu, yoksa LSD radix sort mu kullanmalı?

> **Çözüm:** LSD radix sort. Karşılaştırmalı sıralama yaklaşık $85 \times 10^6 \times \log_2(85 \times 10^6) \approx 2{,}24$ milyar adım ister. LSD, her hane için bir counting sort yaparak $11 \times 85 \times 10^6 = 935$ milyon adımda bitirir. Hoca bu çarpımı 850 milyon diye söyledi; doğrusu 935 milyondur.

5. (derste sözlü, 01:02:40) On iki kelimelik MSD örneğinde (ace, add, bad, bed, bee, cab, dab, dad, ebb, fad, fed, fee) kaç karaktere bakıldı?

> **Çözüm:** Hoca 8 kelime ve 24 karakter üzerinden 19 karaktere (yaklaşık %80) bakıldığını söyledi. Örnekte aslında 12 kelime ve 36 karakter vardır.
>
> **Çözüm (derste verilmedi):** Tek elemanlı gruplara girilmediği varsayılınca 1. harfte 12, 2. harfte 10, 3. harfte 6 karaktere bakılır. Toplam 28 karakterdir, yani yaklaşık %78. 8 karaktere hiç bakılmaz. Bu sayı python ile algoritma çalıştırılarak doğrulandı.

Dersin Toparlaması

Bu derste sıralamanın her zaman elemanları ikişer ikişer karşılaştırarak yapılmak zorunda olmadığı gösterildi. Counting sort, değerlerin frekanslarını sayıp birikimli toplama (dağılıma) çevirerek her elemanın çıktıdaki yerini bulur. Hiç `if` kullanmadan $O(n + k)$ sürede sıralar, sondan başa yerleştirdiği için kararlıdır, ama sayaç dizisi ve ayrı bir çıktı dizisi gerektirir. Comparison counting aynı fikri karşılaştırmayla, $O(n^2)$ sürede uygular. Distribution counting ise counting sort'un $[l, u]$ aralığına kaydırılmış halidir. Radix sort her basamakta kararlı bir counting sort yapar. LSD sağdan sola gider ve sabit uzunluklu anahtarlar (kimlik numarası, IBAN, GUID) için idealdir. MSD soldan sağa gider, değişken uzunluklu girdilerde karakterlerin büyük kısmına bakmadan sıralayabilir ve paralelleştirilebilir, ama küçük parçalarda yavaşlar.

Dersin son kısmında sıralama algoritmaları performans, ek bellek (yerinde ya da yerinde olmayan) ve kararlılık ölçütleriyle karşılaştırıldı ve tasarım yaklaşımlarına göre gruplandı. Hocanın vardığı sonuç, sıralama algoritması seçiminin bir ödünleşim olduğuydu. Bu ders, 14 haftalık teorik derslerin sonuncusuydu. Merge sort ve quicksort'un ayrıntıları algoritma analizi dersine bırakıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.